

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Avinash K	Reg. No.:	192411044
Section / Batch:	CSE	Date:	5/8/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

An online auction platform continuously updates bids and needs to rank items dynamically based on price. Analyze how Quick Sort can be applied for real-time ranking. Identify issues such as pivot selection and uneven data distribution. Apply the Divide and Conquer principle to explain the sorting process. Analyze performance under different cases and propose optimizations, justifying their effectiveness.

Code of Conduct

I Avinash K(192411044) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Avinashk

ASSESSMENT-CO3/AT2-CASE STUDY

← Back

Language: Python C C++ Java

QUESTIONS

Q1

Online Auction Ranking System (Quick Sort)

100 marks

1/1 answered

Q1 Online Auction Ranking System (Quick Sort) 100 marks

An online auction platform continuously updates bids and needs to rank items dynamically based on price. Analyze how Quick Sort can be applied for real-time ranking. Identify issues such as pivot selection and uneven data distribution. Apply divide and conquer principles to explain the sorting process. Analyze performance under different cases and propose optimizations, justifying their effectiveness.

Your Code

```

1 import random
2 from typing import List, Tuple
3
4 def partition(items: List[Tuple[str, int]], low: int, high: int) -> int:
5     pivot_idx = random.randint(low, high)
6     items[pivot_idx], items[high] = items[high], items[pivot_idx]
7     pivot_price = items[high][1]
8
9     i = low - 1
10    for j in range(low, high):
11        if items[j][1] > pivot_price:
12            i += 1
13            items[i], items[j] = items[j], items[i]
14
15    items[i + 1], items[high] = items[high], items[i + 1]
16    return i + 1
17
18 def quick_sort(items: List[Tuple[str, int]], low: int, high: int) -> None:
19     if low < high:
20         p = partition(items, low, high)
21         quick_sort(items, low, p - 1)
22         quick_sort(items, p + 1, high)
23
24

```

Input

stdin input

Output

```

1. Laptop - 1200
2. Camera - 900
3. phone - 800
4. Tablet - 650
5. headphones - 300

```

1. Introduction

Online auction platforms such as eBay require items to be ranked instantly according to the highest bid. Every time a new bid is placed, the ranking should be updated efficiently. Quick Sort is a Divide and Conquer algorithm that sorts data quickly on average, making it suitable for dynamic ranking when periodic sorting is required.

2. Problem Analysis

The auction system faces several challenges:

- Continuous arrival of new bids.
- Thousands of auction items.
- Fast ranking updates.
- Avoiding delays in displaying rankings.
- Efficient memory usage.

Example

Item	Bid Price
Laptop	1200
Phone	800
Camera	900
Tablet	650
Headphones	300
Expected Ranking	

1. Laptop – 1200
 2. Camera – 900
 3. Phone – 800
 4. Tablet – 650
 5. Headphones – 300
-

3. Divide and Conquer Principle

Quick Sort follows three main steps.

Divide

Choose a pivot element.

Example

Pivot = Camera (900)

Conquer

Split the list into two groups.

Greater than Pivot

- Laptop (1200)

Less than Pivot

- Phone (800)
- Tablet (650)
- Headphones (300)

Combine

Recursively sort both groups and combine.

Final Result

Laptop → Camera → Phone → Tablet → Headphones

4. Sample Python Code

```
import random
```

```
def partition(arr, low, high):
    pivot_index = random.randint(low, high)
    arr[pivot_index], arr[high] = arr[high], arr[pivot_index]

    pivot = arr[high][1]
    i = low - 1

    for j in range(low, high):
        if arr[j][1] > pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i+1], arr[high] = arr[high], arr[i+1]
    return i+1

def quicksort(arr, low, high):
    if low < high:
        p = partition(arr, low, high)
```

```
quicksort(arr, low, p-1)
quicksort(arr, p+1, high)
```

```
items = [
    ("Laptop",1200),
    ("Phone",800),
    ("Camera",900),
    ("Tablet",650),
    ("Headphones",300)
]
```

```
quicksort(items,0,len(items)-1)
```

```
print("Auction Ranking")
```

```
for i,item in enumerate(items,1):
    print(i,".",item[0],"-",item[1])
```

5. Output

Auction Ranking

1. Laptop - 1200
2. Camera - 900
3. Phone - 800
4. Tablet - 650
5. Headphones - 300

6. Key Issues

1. Pivot Selection

Choosing the first or last element as the pivot repeatedly can create unbalanced partitions, increasing recursive calls and execution time, which may lead to worst-case performance.

2. Uneven Data Distribution

If the data is already sorted or nearly sorted, one partition becomes very large while the other becomes very small, causing the algorithm to degrade to $O(n^2)$.

Example

```
1200
1100
1000
900
800
```

3. Frequent Bid Updates

Every new bid changes the ranking. Running Quick Sort after every single update may be inefficient for systems with continuous bidding.

4. Large Dataset

Auction platforms may contain millions of items. Recursive calls increase processing time and may consume additional stack memory.

5. Duplicate Bid Values

Many users may place identical bids. Standard partitioning may perform unnecessary comparisons when handling duplicate values.

7. Performance Analysis

Case	Time Complexity
------	-----------------

Best Case	$O(n \log n)$
-----------	---------------

Average Case	$O(n \log n)$
--------------	---------------

Worst Case	$O(n^2)$
------------	----------

Space Complexity	$O(\log n)$
------------------	-------------

Best Case

Balanced partitions produce efficient recursive calls.

Example

900

600 1200

300 700 1000 1400

Average Case

A randomly selected pivot usually creates reasonably balanced partitions, resulting in good performance.

Worst Case

If the pivot is always the smallest or largest element, the partitions become highly unbalanced.

Example

100

200

300

400

500

Each recursive call processes only one element, resulting in $O(n^2)$ time complexity.

8. Proposed Optimizations

1. Random Pivot

Randomly selecting the pivot reduces the probability of worst-case partitions and improves average performance.

2. Median-of-Three Pivot

Selecting the median of the first, middle, and last elements provides a better pivot and creates more balanced partitions.

3. Hybrid Quick Sort

For small subarrays, switch to Insertion Sort. This reduces recursion overhead and improves execution speed.

4. Three-Way Partitioning

Separate elements into three groups:

- Less than pivot
- Equal to pivot
- Greater than pivot

This efficiently handles duplicate bid values.

5. Parallel Quick Sort

Sort the left and right partitions simultaneously using multiple processors or threads, improving performance for

very large datasets.

9. Justification of Solutions

The proposed optimizations improve Quick Sort's efficiency and scalability.

- Random Pivot reduces the likelihood of worst-case partitions.
- Median-of-Three produces better pivot selection.
- Hybrid Quick Sort improves performance on small datasets.
- Three-Way Partitioning efficiently handles duplicate bid values.
- Parallel Quick Sort speeds up sorting for large-scale auction systems.

These techniques enable faster ranking updates and better overall system performance.

10. Conclusion

Quick Sort is an efficient Divide and Conquer algorithm for ranking auction items based on bid prices. It offers $O(n \log n)$ average-case performance and is suitable for real-time auction platforms. However, poor pivot selection and uneven data distribution can reduce performance to $O(n^2)$. Using techniques such as Random Pivot, Median-of-Three Pivot, Hybrid Quick Sort, Three-Way Partitioning, and Parallel Processing significantly improves efficiency, making Quick Sort a practical solution for handling large numbers of continuously changing bids.

Code:

```
import random
```

```
def partition(items, low, high):
```

```
    pivot_index = random.randint(low, high)
```

```
    items[pivot_index], items[high] = items[high], items[pivot_index]
```

```
    pivot = items[high][1]
```

```
    i = low - 1
```

```
    for j in range(low, high):
```

```
        if items[j][1] > pivot:    # Descending order
```

```
            i += 1
```

```
            items[i], items[j] = items[j], items[i]
```

```
    items[i + 1], items[high] = items[high], items[i + 1]
```

```
    return i + 1
```

```
def quick_sort(items, low, high):
```

```
    if low < high:
```

```
        pi = partition(items, low, high)
```

```
        quick_sort(items, low, pi - 1)
```

```
        quick_sort(items, pi + 1, high)
```

```
# Input
```

```
n = int(input("Enter number of auction items: "))
```

```
items = []
```

```
for i in range(n):
```

```
    name = input("Enter item name: ")
```

```
    price = int(input("Enter bid price: "))
```

```
    items.append((name, price))
```

```
quick_sort(items, 0, len(items) - 1)
```

```
print("\nAuction Ranking:")
```

```
for i, (name, price) in enumerate(items, 1):
```

```
    print(f'{i}. {name} - {price}')
```

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				

Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____